

최신 주식 정보를
알려 주는 AI 투자자

최신 주식 정보를 알려 주는 AI 투자자

1. 평션 콜링의 기초
2. GPT와 미국 주식 이야기하기
3. 스트림 출력하기

1. 평션 콜링의 기초

- GPT야, 지금 몇 시지?

- 03-2절 'GPT와 멀티턴 대화하기'에서 만든 챗봇(multi_turn.py)에 질문하기

사용자: **지금 몇 시야?**

AI: 지금 시각이 필요하군요. 제 쪽에서는 실시간 시각을 직접 확인할 수 없어요.

원하시면

- 당신의 도시나 시간대를 알려주시면 그 지역 기준으로 현재 시각을 계산해 드릴게요.
- 아니면 바로 지금 당신 기기의 시계로 확인하시는 게 가장 빠릅니다. 예: 스마트폰의 시계 앱이나 화면 상단의 시계.

- GPT 같은 언어 모델은 아무리 똑똑해 보이더라도 기본적으로 이전 텍스트를 기반으로 다음에 나올 문장을 예측하는 모델
- 현재 시간이 몇 시인지, 밖의 날씨가 어떤지, 최근에 유행하는 음악은 무엇인지 알지 못함

1. 평션 콜링의 기초

- 평션 콜링이란?

- 평션 콜링(Function calling)을 활용하면
 - GPT에게 함수와 그에 관한 설명을 제공하고 상황에 맞는 특정 함수를 호출하도록 할 수 있음
 - GPT는 함수 실행 결과를 해석하여 사용자에게 답변을 제공함
- 오픈AI에서는 평션 콜링 기능에 사용할 수 있는 함수 담은 도구 목록을 딕셔너리 형태로 정의
 - 도구 목록의 딕셔너리는 GPT 모델이 어떤 도구를 사용할 수 있는지 알려 주는 설명서 역할
 - GPT API를 호출할 때 이 도구 목록도 함께 전달됨

1. 평선 콜링의 기초

- (실습) 평선 콜링 적용하기 - 지금이 몇 시인지 GPT가 답할 수 있도록 구현

1. 파이썬 파일 gpt_functions_0.py을 생성하여 현재 시간 파악하기

```
from datetime import datetime

def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(now)
    return now

if __name__ == '__main__':
    get_current_time()
```

1. 평션 콜링의 기초

- (실습) 평션 콜링 적용하기

2. 현재 시간을 알려 주는 함수를 평션 콜링 기능으로 활용하기 위한 설명 추가

```
from datetime import datetime

def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(now)
    return now

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "현재 날짜와 시간을 반환합니다.",
        }
    },
]

if __name__ == '__main__':
    get_current_time()
```

1. 평션 콜링의 기초

- (실습) 평션 콜링 적용하기

3. 새 파일 time_terminal_0.py를 만들어 평션 콜링에 사용할 도구 목록인 tools를 포함하도록 수정

```
from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def get_ai_response(messages, tools=None):
    response = client.chat.completions.create(
        model="gpt-5-nano", # 응답 생성에 사용할 모델 지정
        messages=messages, # 대화 기록을 입력으로 전달
        tools=tools, # 사용 가능한 도구 목록 전달
    )
    return response # 생성된 응답 내용 반환
```

1. 평션 콜링의 기초

- (실습) 평션 콜링 적용하기

4. 터미널 창에서 반복적으로 호출하기 위해 get_ai_response 함수를 while True: 내부에서 사용

```
... ( 생략 ) ...

messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
]

while True:
    user_input = input("사용자\tt: ") # 사용자 입력 받기

    if user_input == "exit": # 사용자가 대화를 종료하려는지 확인
        break

    messages.append({
        "role": "user",
        "content": user_input
    }) # 사용자 메시지 대화 기록에 추가

    # 2. AI 응답 받기
    ai_response = get_ai_response(messages, tools=tools)
    ai_message = ai_response.choices[0].message
    print(ai_message) # gpt에서 반환되는 값을 파악하기 위해 임시로 추가

... ( 생략 ) ...
```


1. 평션 콜링의 기초

- (실습) 평션 콜링 적용하기

5. 결과

사용자 : 지금 몇 시야?

```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_ID6eoXTLyRHEYvn3POFtewcB', function=Function(arguments='{"timezone":"Asia/Seoul"}', name='get_current_time'), type='function')])
```

2026-04-01 20:09:23 Asia/Seoul

AI : 현재 시각은 20:09:23입니다. (한국 표준시, Asia/Seoul 기준) 오늘은 2026년 4월 1일이에요. 필요하면 더 자세히 알려드릴게요.

1. 펄션 콜링의 기초

- (실습) 뉴욕은 지금 몇 시야?

- GPT에게 다짜고짜 지금 뉴욕은 몇 시냐고 물어보면 잘못된 답변을 함

사용자 : 뉴욕은 지금 몇시야?

```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_om2lkwMojLCaxfGwa0LrkgFv', function=Function(arguments='{"timezone":"America/New_York"}', name='get_current_time'), type='function')])
```

2026-04-02 17:12:18 Asia/Seoul

AI : 뉴욕은 현재 대략 새벽 4시 12분(EDT)쯤입니다. 2026-04-02 기준으로 동부 표준시가 아닌 일광 절약 시간 EDT로 계산됩니다.

정확히 맞춰 알람이나 일정으로 설정해 드릴까요? 또 다시 확인해 드릴 수도 있습니다.

- 파이썬에서는 지역별로 다른 시간대(타임존)에 맞는 시간 정보를 얻기 위해 pytz 라이브러리를 제공
(예) 'Asia/Seoul', 'America/New_York'

1. 평션 콜링의 기초

- (실습) 도시별 시간 알려 주기

1. 앞에서 만든 gpt_functions_0.py 파일 수정하고, gpt_functions.py로 저장

```
from datetime import datetime
import pytz

def get_current_time(timezone: str = 'Asia/Seoul'):
    try:
        timezone = timezone.strip() # 공백 제거
        tz = pytz.timezone(timezone)
    except Exception:
        return f"잘못된 타임존입니다: {timezone}"

    now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    now_timezone = f"{now} {timezone}"
    print(now_timezone)
    return now_timezone
```

1. 평션 콜링의 기초

- (실습) 도시별 시간 알려 주기

2. GPT가 사용할 수 있는 도구를 담은 tools 리스트에서 get_current_time 함수의 항목을 수정

... (생략) ...

```
tools = [  
  {  
    "type": "function",  
    "function": {  
      "name": "get_current_time",  
      "description": "주어진 타임존(예: Asia/Seoul, Asia/Tokyo, America/New_York)의 현재 날짜와 시간을 반환합니다.",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "timezone": {  
            "type": "string",  
            "description": "현재 날짜와 시간을 반환할 타임존을 입력하세요. (예: Asia/Seoul)",  
          },  
        },  
        "required": ['timezone'],  
      },  
    },  
  },  
],
```

... (생략) ...

1. 평션 콜링의 기초

- (실습) 도시별 시간 알려 주기

3. time_terminal_0.py에 앞서 추가한 타임존 정보를 반영하고, time_terminal_1.py로 저장

```
... ( 생략 ) ...

while True:
    user_input = input("사용자\t: ")

    if user_input.lower() == "exit":
        break

    # 1. 사용자 메시지 추가
    messages.append({
        "role": "user",
        "content": user_input
    })

    # 2. 1차 응답
    ai_response = get_ai_response(messages, tools=tools)
    ai_message = ai_response.choices[0].message

    tool_calls = ai_message.tool_calls

... ( 생략 ) ...
```

1. 평션 콜링의 기초

- (실습) 도시별 시간 알려 주기

4. 결과 확인

사용자 : 지금 몇시야?

ChatCompletionMessage(content='어느 시간대의 시간을 알고 싶으신가요? 예를 들어, 서울 시간(Asia/Seoul), 도쿄 시간(Asia/Tokyo), 뉴욕 시간(America/New_York) 등을 말씀해 주세요.', refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=None)

AI : 어느 시간대의 시간을 알고 싶으신가요? 예를 들어, 서울 시간(Asia/Seoul), 도쿄 시간(Asia/Tokyo), 뉴욕 시간(America/New_York) 등을 말씀해 주세요.

사용자 : 서울

ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_fPVIbaEfrWrZiukycBs9NiEA', function=Function(arguments='{"timezone":"Asia/Seoul"}', name='get_current_time'), type='function')])

2026-04-24 22:04:51 Asia/Seoul

AI : 현재 서울 시간은 2026년 4월 24일 오후 10시 4분입니다.

사용자 : 뉴욕은?

ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_C8F8nMpgadoliCyuAucQalWQ', function=Function(arguments='{"timezone":"America/New_York"}', name='get_current_time'), type='function')])

2026-04-24 09:05:05 America/New_York

AI : 현재 뉴욕 시간은 2026년 4월 24일 오전 9시 5분입니다.

사용자 : exit

1. 펄션 콜링의 기초

- (실습) 여러 도시의 시간을 한 번에 대답할 수 있게 하기

1. 앞선 실습에서 만든 챗봇에 한 번에 여러 도시의 시간을 물어보면 명확한 답을 주지 못하는 경우 발생

사용자 : 서울, 뉴욕, 런던, 파리는 몇시야?

```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_wPctrrfh4DrqKJOEE1e1Gorz',
function=Function(arguments='{"timezone": "Asia/Seoul"}', name='get_current_time'), type='function'),
ChatCompletionMessageFunctionToolCall(id='call_WgNydKiYEInt5jnxluZbKw3K', function=Function(arguments='{"timezone":
"America/New_York"}', name='get_current_time'), type='function'),
ChatCompletionMessageFunctionToolCall(id='call_gaRkaFzmocI7M5gsWXkFU9or', function=Function(arguments='{"timezone":
"Europe/London"}', name='get_current_time'), type='function'),
ChatCompletionMessageFunctionToolCall(id='call_G40sLf1YEcE9SRFKROVvtUkG', function=Function(arguments='{"timezone":
"Europe/Paris"}', name='get current time'), type='function'))]
```

2026-04-24 22:08:31 Asia/Seoul

Traceback (most recent call last):

```
File "c:\ai agent\src\except chap05\chap07\sec01\time terminal 1.py", line 62, in <module>
```

```
ai_response = get_ai_response(messages, tools=tools) # 다시 GPT 응답 받기
```

AA

... (생략) ...

- 해결책: GPT가 함수를 여러 번 호출할 수 있도록 해줘야 함

1. 평션 콜링의 기초

- (실습) 여러 도시의 시간을 한 번에 대답할 수 있게 하기

2. time_terminal_1.py에 코드를 수정하고, time_terminal.py로 저장

... (생략) ...

```
if tool_calls: # tool_calls가 있는 경우
    for tool_call in tool_calls:
        tool_name = tool_call.function.name # 실행해야한다고 판단한 함수명 받기
        tool_call_id = tool_call.id # tool_call 아이디 받기
        arguments = json.loads(tool_call.function.arguments) # (1) 문자열을 딕셔너리로 변환

        if tool_name == "get_current_time": # ⑤ 만약 tool_name이 "get_current_time"이라면
            tool_result = get_current_time(timezone=arguments['timezone'])

            messages.append({
                "role": "tool", # role을 "tool"으로 설정
                "tool_call_id": tool_call_id,
                "name": tool_name,
                "content": tool_result, # 타임존 추가
            })
```

... (생략) ...

1. 평션 콜링의 기초

- (실습) 여러 도시의 시간을 한 번에 대답할 수 있게 하기

3. 결과 확인

사용자 : 서울, 뉴욕, 런던, 파리 시간 알려줘

```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_CEDzHy1TeuPAvpRDI8wh94hi', function=Function(arguments='{"timezone": "Asia/Seoul"}',
name='get_current_time'), type='function'), ChatCompletionMessageFunctionToolCall(id='call_TbOcGCXkkzP5y5JIYqopujai',
function=Function(arguments='{"timezone": "America/New_York"}', name='get_current_time'), type='function'),
ChatCompletionMessageFunctionToolCall(id='call_VqQ2K7xLsqMUxWYHkwh9VooS', function=Function(arguments='{"timezone": "Europe/London"}',
name='get_current_time'), type='function'), ChatCompletionMessageFunctionToolCall(id='call_6ii3iYtcGZemNpKzZjRi0yDO',
function=Function(arguments='{"timezone": "Europe/Paris"}', name='get_current_time'), type='function'))]
```

2026-04-24 22:21:58 Asia/Seoul

2026-04-24 09:21:58 America/New_York

2026-04-24 14:21:58 Europe/London

2026-04-24 15:21:58 Europe/Paris

AI : 다음은 요청하신 도시의 현재 시간입니다.

- 서울: 2026-04-24 22:21:58 (Asia/Seoul)

- 뉴욕: 2026-04-24 09:21:58 (America/New_York)

- 런던: 2026-04-24 14:21:58 (Europe/London)

- 파리: 2026-04-24 15:21:58 (Europe/Paris)

원하시면 다른 형식으로도 변환해 드리거나, 더 많은 도시의 시간도 알려드릴게요.

1. 평션 콜링의 기초

- (실습) 스트림릿에서 평션 콜링 사용하기

- 앞에서 만든 챗봇을 웹 브라우저에서 사용할 수 있도록 스트림릿으로 수정해 보기

1. 스트림릿을 импорт

```
from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st
```

... (생략) ...

1. 평션 콜링의 기초

- (실습) 스트림릿에서 평션 콜링 사용하기

2. streamlit 초기 시스템 메시지 설정하기

```
... ( 생략 ) ...

def get_ai_response(messages, tools=None):
    ... ( 생략 ) ...

# 세션 상태 초기화
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
    ]

# UI 제목
st.title("💬 상담 챗봇")

# 기존 대화 출력
for msg in st.session_state.messages:
    if msg["role"] == "user":
        st.chat_message("user").write(msg["content"])
    elif msg["role"] == "assistant":
        st.chat_message("assistant").write(msg["content"])

... ( 생략 ) ...
```

1. 평션 콜링의 기초

- (실습) 스트림릿에서 평션 콜링 사용하기

3. 사용자 입력 처리하기

```
... ( 생략 ) ...

# 사용자 입력
if user_input := st.chat_input("메시지를 입력하세요"):
    # 1. 사용자 메시지 추가
    st.session_state.messages.append({
        "role": "user",
        "content": user_input
    })
    st.chat_message("user").write(user_input)

    # 2. 1차 응답
    ai_response = get_ai_response(st.session_state.messages, tools=tools)
    ai_message = ai_response.choices[0].message

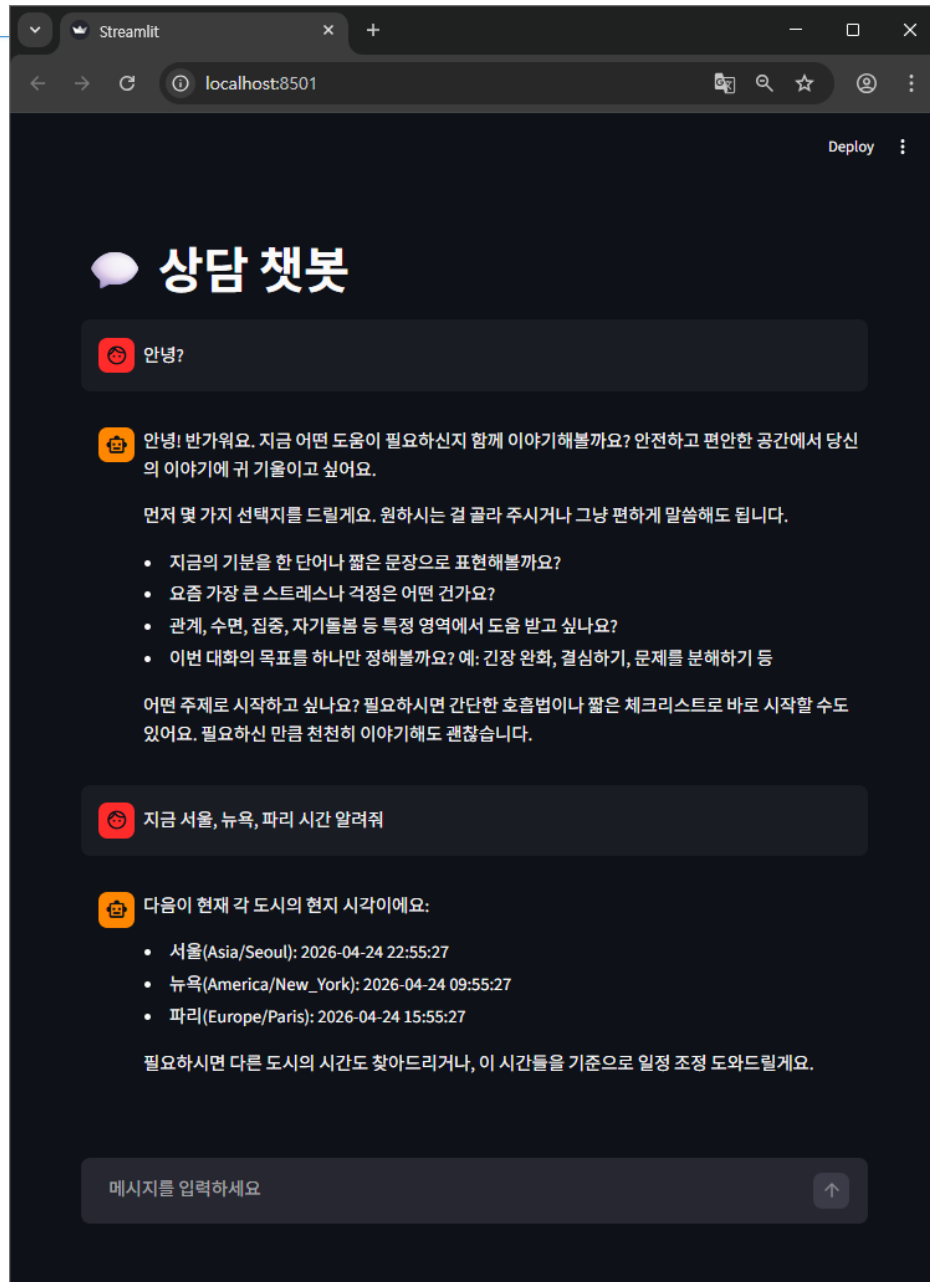
    # tool call 포함해서 저장
    st.session_state.messages.append({
        "role": "assistant",
        "content": ai_message.content,
        "tool_calls": ai_message.tool_calls
    })

... ( 생략 ) ...
```

1. 평션 콜링의 기초

- (실습) 스트림릿에서 평션 콜링 사용하기

4. 터미널 창에서 `streamlit run 파일명 .py`을 입력해 실행



2. GPT와 미국 주식 이야기하기

- yfinance 사용하기

- ‘요새 마이크로소프트 주식 얼마야?’, ‘테슬라 주식을 지금 더 사야 할까? 아님 팔아야 할까?’등의 대화를 하려면 GPT가 최신 주식 정보를 알 수 있어야 함
- yfinance라는 파이썬 오픈소스 라이브러리를 사용하면 미국 주식 정보를 쉽게 가져올 수 있음
- yfinance는 야후 파이낸스(Yahoo Finance)의 금융 데이터를 쉽게 가져올 수 있게 해주는 오픈소스 파이썬 라이브러리
- 주가, 재무제표, 거래량 등 다양한 데이터를 데이터프레임 형태로 가져올 수 있음

2. GPT와 미국 주식 이야기하기

- (실습) yfinance 사용하기

1. 터미널 창에 다음과 같이 입력해 가상 환경에 yfinance 설치

```
(.venv)  
$ pip install yfinance
```

2. yfinance.ipynb 파일을 생성하고, 첫 번째 셀에 다음 코드를 입력하고 실행

```
import yfinance as yf  
  
# Microsoft (MSFT)에 대한 Ticker 객체 생성  
msft = yf.Ticker("MSFT")  
  
# Ticker 객체에 대한 정보 출력 (.py에서 실행할 때는 print(msft.info)로 사용)  
display(msft.info)
```

2. GPT와 미국 주식 이야기하기

- (실습) yfinance 사용하기

3. 코드를 실행하면 msft.info가 실행되며
마이크로소프트의 기본 정보가 출력됨

```
{'address1': 'One Microsoft Way',  
  'city': 'Redmond',  
  'state': 'WA',  
  'zip': '98052-6399',  
  'country': 'United States',  
  'phone': '425 882 8080',  
  'website': 'https://www.microsoft.com',  
  'industry': 'Software - Infrastructure',  
  'industryKey': 'software-infrastructure',  
  'industryDisp': 'Software - Infrastructure',  
  'sector': 'Technology',  
  'sectorKey': 'technology',  
  'sectorDisp': 'Technology',  
  'longBusinessSummary': 'Microsoft Corporation de  
  'fullTimeEmployees': 228000,  
  'companyOfficers': [{ 'maxAge': 1,  
    'name': 'Mr. Satya Nadella',  
    'age': 58,  
    'title': 'Chairman & CEO',  
    'yearBorn': 1967,  
    'fiscalYear': 2025,  
    'totalPay': 12251294,  
    'exercisedValue': 0,  
    'unexercisedValue': 0},  
    { 'maxAge': 1,  
  ...  
  'cryptoTradeable': False,  
  'shortName': 'Microsoft Corporation',  
  'longName': 'Microsoft Corporation',  
  'displayName': 'Microsoft',  
  'trailingPegRatio': 1.3104}
```


2. GPT와 미국 주식 이야기하기

- (실습) yfinance 사용하기

4. 최근 5일간의 주가 정보 확인

```
hist = msft.history(period="5d") # 5일간의 주가 데이터를 가져옴
display(hist) # 데이터 출력
```

	Open	High	Low	Close	Volume	Dividends	Stock Splits
Date							
2026-04-20 00:00:00-04:00	421.149994	423.329987	416.299988	418.070007	27582200	0.0	0.0
2026-04-21 00:00:00-04:00	420.239990	427.179993	417.200012	424.160004	32048500	0.0	0.0
2026-04-22 00:00:00-04:00	426.190002	433.700012	423.670013	432.920013	29378200	0.0	0.0
2026-04-23 00:00:00-04:00	419.890015	423.660004	411.410004	415.750000	38151200	0.0	0.0
2026-04-24 00:00:00-04:00	416.984985	421.619995	415.799988	417.994995	5716480	0.0	

열 이름	설명
Open(시가)	거래 시작 시점의 주가
High(고가)	해당 거래일 동안의 최고 가격
Low(저가)	해당 거래일 동안의 최저 가격
Close(종가)	해당 거래일 마지막 시점의 주가
Volume(거래량)	해당 거래일 동안 거래된 주식의 총 수량
Dividends(배당금)	주식 배당금
Stock Splits(주식 분할)	해당 거래일에 발생한 주식 분할 비율, 없으면 0으로 표기

2. GPT와 미국 주식 이야기하기

- (실습) yfinance 사용하기

5. 해당 종목에 대한 애널리스트들의 분석 결과도 찾아볼 수 있음

```
msft.recommendations # 추천 정보 출력
```

	period	strongBuy	buy	hold	sell	strongSell
0	0m	10	43	3	0	0
1	-1m	10	45	3	0	0
2	-2m	10	44	3	0	0
3	-3m	12	45	1	0	0

- period: 분석가들이 추천한 시점(0m: 현재, -1m: 1개월 전, -2m: 2개월 전, -3m: 3개월 전을 의미)
- 추천은 strongBuy부터 strongSell까지 5개의 등급으로 분류

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- GPT가 최신 주가 정보에 기반하여 답변하도록 하려면 필요한 기능을 함수로 만들어 평션 콜링을 활용해야 함

- 회사 기본 정보 가져오기

1. 회사 기본 정보를 가져오는 get_yf_stock_info 함수 만들기

```
from datetime import datetime
import pytz
import yfinance as yf

... ( 생략 ) ...

def get_yf_stock_info(ticker: str):
    try:
        stock = yf.Ticker(ticker)
        info = stock.info
        #print(info)
        return str(info) # GPT에게 전달하기 위해 문자열로 변환. 토큰의 크기가 클 수 있어 주의
        # return str({
        #     "symbol": info.get("symbol"),
        #     "shortName": info.get("shortName"),
        #     "currentPrice": info.get("currentPrice"),
        #     "marketCap": info.get("marketCap"),
        # })
    except Exception as e:
        return f"주식 정보를 가져오는 중 오류 발생: {str(e)}"

... ( 생략 ) ...
```

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- 회사 기본 정보 가져오기

- 2. 코드를 실행하면 애플의 정보가 출력됨

```
{'address1': 'One Apple Park Way', 'city': 'Cupertino', 'state': 'CA', 'zip': '95014', 'country': 'United States', 'phone': '(408) 996-1010', 'website': 'https://www.apple.com', 'industry': 'Consumer Electronics', 'industryKey': 'consumer-electronics', 'industryDisp': 'Consumer Electronics', 'sector': 'Technology', 'sectorKey': 'technology', 'sectorDisp': 'Technology', 'longBusinessSummary': 'Apple Inc. designs, manufactures, and markets smartphones, personal computers, tablets, wearables, and accessories worldwide. The company offers iPhone, a line of smartphones; Mac, a line of personal computers; iPad, a line of multi-purpose tablets; and wearables, home, and accessories comprising AirPods, Apple Vision Pro, Apple TV, Apple Watch, Beats products, and HomePod, as well as Apple branded and third-party accessories. It also provides AppleCare support and cloud services; and operates various platforms, including the App Store that allow customers to discover and download applications and digital content, such as books, music, video, games, and podcasts, as well as advertising services include third-party licensing arrangements and its own advertising platforms.
```

... (생략) ...

```
'earningsTim'averageAnalystRating': '1.9 - Buy', 'cryptoTradeable': False, 'shortName': 'Apple Inc.', 'dividendDate': 1770854400, 'earningsTimestamp': 1769720400, 'earningsTimestampStart': 1777579200, 'earningsTimestampEnd': 1777579200, 'earningsCallTimestampStart': 1769724000, 'earningsCallTimestampEnd': 1769724000, 'isEarningsDateEstimate': True, 'epsTrailingTwelveMonths': 7.9, 'epsForward': 9.31526, 'epsCurrentYear': 8.50708, 'priceEpsCurrentYear': 30.04909, 'fiftyDayAverageChange': -4.5675964, 'fiftyDayAverageChangePercent24000', 'earningsCallTimestampEnd': 1769724000, 'isEarningsDateEstimate': True, 'epsTrailingTwelveMonths': 7.9, 'epsForward': 9.31526, 'epsCurrentYear': 8.50708, 'priceEpsCurrentYear': 30.04909, 'fiftyDayAverageChange': -4.5675964, 'fiftyDayAverageChangePercent26', 'epsCurrentYear': 8.50708, 'priceEpsCurrentYear': 30.04909, 'fiftyDayAverageChange': -4.5675964, 'fiftyDayAverageChangePercent': -0.017554337, 'twoHundredDayAverageChange': 6.769104, 'twoHundredDayAverageChangePercent': 0.027200352, 'sourceInterval': 15, 'exchangeDataDelayedBy': 0, 'displayName': 'Apple', 'trailingPegRatio': 2.2707}
```

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- 회사 기본 정보 가져오기

3. get_yf_stock_info 함수의 설명을 tools에 추가하여 GPT에서 이 함수를 사용할 수 있도록 하기

```
tools = [  
    ... ( 생략 ) ...  
    {  
        "type": "function",  
        "function": {  
            "name": "get_yf_stock_info",  
            "description": "주어진 티커(예: AAPL)의 현재 가격, 시가총액 등 핵심 Yahoo Finance 정보를 반환합니다.",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    'ticker': {  
                        'type': 'string',  
                        'description': 'Yahoo Finance 정보를 반환할 종목의 티커를 입력하세요. (예: AAPL)',  
                    },  
                },  
                "required": ['ticker'],  
            },  
        },  
    },  
    ... ( 생략 ) ...  
]
```

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- 회사 기본 정보 가져오기

4. get_yf_stock_info도 사용할 수 있도록 추가

```
... ( 생략 ) ...

tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls를 가져옵니다.
if tool_calls:
    for tool_call in tool_calls:
        tool_name = tool_call.function.name
        tool_call_id = tool_call.id
        arguments = json.loads(tool_call.function.arguments)

        if tool_name == "get_current_time":
            timezone = arguments.get("timezone", "Asia/Seoul").strip()
            func_result = get_current_time(timezone=timezone)

        elif tool_name == "get_yf_stock_info":
            ticker = arguments.get("ticker", "").upper()
            func_result = get_yf_stock_info(ticker=ticker)

        st.session_state.messages.append({
            "role": "tool", "tool_call_id": tool_call_id,
            "name": tool_name, "content": func_result,
        })
... ( 생략 ) ...
```

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- 회사 기본 정보 가져오기

5. streamlit run 파일명 .py 명령어를 터미널 창에 입력해 코드를 실행하고, 챗봇에게 테슬라에 대해 물어보기

 **상담 챗봇**

 테슬라 회사에 대해 알려줘

 다음은 테슬라(Tesla, Inc.)에 대한 간단한 개요입니다.

- 기본 정보
 - 미국의 전기차(EV) 및 에너지 기업으로, 세계 EV 시장의 선두주자 중 하나입니다.
 - 설립: 2003년. 주요 창립자 중 하나였던 Elon Musk가 현재도 CEO로 활동 중입니다.
 - 본사 및 운영 거점: 텍사스 주 오스틴에 위치한 본사와 캘리포니아 주 프리몬트의 주요 공장을 포함해 전 세계에 생산 시설을 운영합니다.
 - 주식: 나스닥 상장 ticker TSLA.
- 핵심 비전
 - 세계 에너지 전환 가속화: 전기차를 대중화하고, 재생에너지 기반의 발전/저장을 확산시키려는 목표를 가지고 있습니다.
- 주요 사업 영역
 - 자동차(EV): 승용 전기차 모델 S(럭셔리 세단), 모델 3(중형 세단), 모델 X(SUV), 모델 Y(컴팩트 SUV), 사이버트럭, 로드스터 등 다양한 차종을 판매합니다.
 - 에너지 생성/저장: 가정용 배터리 Powerwall, 상용/대형 배터리 솔루션 Powerpack/Megapack, 태양광 패널 및 솔라루프 등 에너지 생산과 저장 솔루션을 제공합니다.

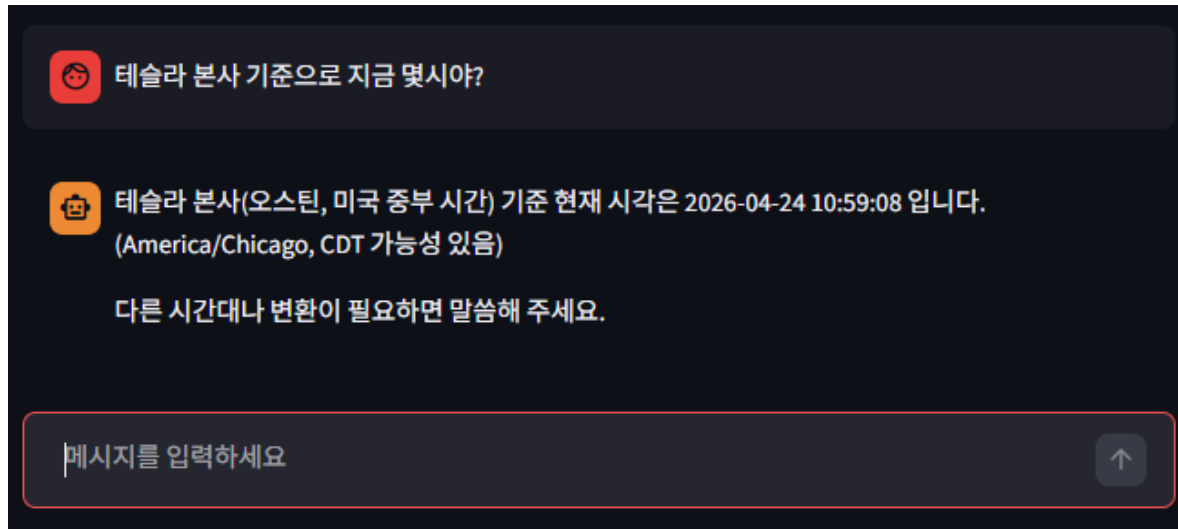
메시지를 입력하세요 

2. GPT와 미국 주식 이야기하기

- (실습) GPT에서 사용할 yfinance 관련 함수 만들기

- 회사 기본 정보 가져오기

6. 테슬라 본사 기준으로 지금 몇 시인지 물어보기



```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_hDpDafJHpRoRDZ959bNZBEGE', function=Function(arguments='{"timezone":"America/Chicago"}', name='get_current_time', type='function'))])
```


2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

1. 최근 주가 기록을 가져오는 함수 `get_yf_stock_history`와 추천 정보를 가져오는 함수 `get_yf_stock_recommendations` 만들기

```
... ( 생략 ) ...

def get_yf_stock_info(ticker: str):
    ... ( 생략 ) ...

def get_yf_stock_history(ticker: str, period: str):
    try:
        ticker = ticker.upper()
        stock = yf.Ticker(ticker)
        history = stock.history(period=period)

        if history.empty:
            return "데이터가 없습니다."

        history = history.tail(10) # 길면 모델 응답 깨질 수 있어 줄임
        history_md = history.to_markdown() # 데이터프레임을 마크다운 형식으로 변환
        #print(history_md)
        return history_md

    except Exception as e:
        return f"주가 데이터를 가져오는 중 오류 발생: {str(e)}"

continue...
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

1. 최근 주가 기록을 가져오는 함수 `get_yf_stock_history`와 추천 정보를 가져오는 함수 `get_yf_stock_recommendations` 만들기

```
def get_yf_stock_recommendations(ticker: str):
    stock = yf.Ticker(ticker)
    recommendations = stock.recommendations
    recommendations_md = recommendations.to_markdown() # 데이터프레임을 마크다운 형식으로 변환
    #print(recommendations_md)
    return recommendations_md

tools = [

... ( 생략 ) ...

]

if __name__ == '__main__':
    # get_current_time('America/New_York')
    #get_yf_stock_info('AAPL')

    get_yf_stock_history('AAPL', '5d')
    print('----')
    get_yf_stock_recommendations('AAPL')
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

2. get_yf_stock_history/get_yf_stock_recommendations 함수 설명을 tools에 추가하여 GPT에서 이 함수를 사용할 수 있도록 하기

```
tools = [  
... (생략) ...  
    {  
        "type": "function",  
        "function": {  
            "name": "get_yf_stock_history",  
            "description": "해당 종목의 Yahoo Finance 주가 정보를 반환합니다.",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    'ticker': {  
                        'type': 'string',  
                        'description': 'Yahoo Finance 주가 정보를 반환할 종목의 티커를 입력하세요. (예: AAPL)',  
                    },  
                    'period': {  
                        'type': 'string',  
                        'description': '주가 정보를 조회할 기간을 입력하세요. (예: 1d, 5d, 1mo, 1y, 5y)',  
                    },  
                },  
                "required": ['ticker', 'period'],  
            },  
        },  
    },  
],  
continue...
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

2. get_yf_stock_history/get_yf_stock_recommendations 함수 설명을 tools에 추가하여 GPT에서 이 함수를 사용할 수 있도록 하기

```
{
  "type": "function",
  "function": {
    "name": "get_yf_stock_recommendations",
    "description": "해당 종목의 Yahoo Finance 추천 정보를 반환합니다.",
    "parameters": {
      "type": "object",
      "properties": {
        "ticker": {
          "type": "string",
          "description": "Yahoo Finance 추천 정보를 반환할 종목의 티커를 입력하세요. (예: AAPL)",
        },
      },
      "required": ["ticker"],
    },
  },
},
]
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

3. 코드를 처음 실행하면 뜨는 오류 메시지

- `.to_markdown()` 메서드를 사용해 데이터프레임을 마크다운 형식으로 변환하는 과정에 필요한 `tabulate` 라이브러리가 현재 파이썬 가상 환경에 설치되어 있지 않아서 발생하는 오류

... (생략) ...

```
in import_optional_dependency raise ImportError(msg)
```

```
ImportError: Missing optional dependency 'tabulate'. Use pip or conda to install tabulate.
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

4. 가상 환경에서 tabulate를 설치/재실행

```
(.venv)  
$ pip install tabulate
```

- 실행 결과

Date		Open	High	Low	Close	Volume	Dividends	Stock Splits
2026-03-27 00:00:00-04:00		253.9	255.49	248.07	248.8	4.79e+07	0	0
2026-03-30 00:00:00-04:00		250.07	250.87	245.51	246.63	3.94462e+07	0	0
2026-03-31 00:00:00-04:00		247.91	255.48	247.1	253.79	4.95981e+07	0	0
2026-04-01 00:00:00-04:00		254.08	256.18	253.33	255.63	4.00001e+07	0	0
2026-04-02 00:00:00-04:00		254.14	254.76	250.65	252.13	3.78987e+06	0	0

	period	strongBuy	buy	hold	sell	strongSell		
0	0m	6	25	15	1	1		
1	-1m	5	25	16	1	1		
2	-2m	6	24	15	1	2		

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

5. GPT API를 사용하는 스트림릿 코드에서도 함수들을 사용할 수 있도록 수정

```
from gpt_functions import get_current_time, tools, get_yf_stock_info, get_yf_stock_history, get_yf_stock_recommendations

... ( 생략 ) ...

    if tool_name == "get_current_time":
        timezone = arguments.get("timezone", "Asia/Seoul").strip()
        func_result = get_current_time(timezone=timezone)

    elif tool_name == "get_yf_stock_info":
        ticker = arguments.get("ticker", "").upper()
        func_result = get_yf_stock_info(ticker=ticker)

    elif tool_name == "get_yf_stock_history":
        ticker = arguments.get("ticker", "").upper()
        period = arguments.get("period", "5d")
        func_result = get_yf_stock_history(ticker=ticker, period=period)

    elif tool_name == "get_yf_stock_recommendations":
        ticker = arguments.get("ticker", "").upper()
        func_result = get_yf_stock_recommendations(ticker=ticker)

    else:
        func_result = f"지원하지 않는 함수: {tool_name}"

... ( 생략 ) ...
```

2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

6. streamlit run 파일명 .py 명령어로 코드를 실행

- 챗봇에 테슬라 주식이 한 달 전과 비교해 어떤 상태인지 묻기

```
ChatCompletionMessage(content=None, refusal=None, role='assistant',
annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_QXp6frHtP3P
cdZqnzP0IeF6D',
function=Function(arguments='{\"ticker\":\"TSLA\",\"period\":\"1mo\"}',
name='get_yf_stock_history'), type='function')])
```

날짜	시가	고가	저가	종가	거래량	외국인	기관
2026-04-13 00:00:00-04:00	350.07	356.35	348.57	352.42	5.36175e+07	0	0
2026-04-14 00:00:00-04:00	357.67	367.63	354.77	364.2	5.99796e+07	0	0
2026-04-15 00:00:00-04:00	366.83	394.65	362.5	391.95	1.1381e+08	0	0
2026-04-16 00:00:00-04:00	393.81	394.06	381.8	388.9	6.35151e+07	0	0
2026-04-17 00:00:00-04:00	395.92	409.28	391.65	400.62	9.064e+07	0	0
2026-04-20 00:00:00-04:00	402.58	406.8	388.33	392.5	6.46039e+07	0	0
2026-04-21 00:00:00-04:00	393	393.95	385.22	386.42	5.05306e+07	0	0
2026-04-22 00:00:00-04:00	387.57	393.01	385.3	387.51	6.23628e+07	0	0
2026-04-23 00:00:00-04:00	375.28	386	368.39	373.72	9.36833e+07	0	0
2026-04-24 00:00:00-04:00	373.26	382.76	370.735	373.32	3.42181e+07	0	0

AT : 요약으로 말씀드리면, 주가 흐름은 이 기간 동안 대체로 상승했습니다.

상담 챗봇

테슬라 주식이 한달전에 비해 어때?

요약으로 말씀드리면, 주가 흐름은 이 기간 동안 대체로 상승했습니다.

- 기준 기간: 4/13 종가 352.42 → 4/24 종가 373.32, 약 +6% 상승
- 기간 중 특징: 4/17에 intraday 고점 409.28까지 갔고, 시작일인 4/13엔 저가대역 348.57이 확인되었습니다.
- 전체적으로는 변동성이 있었지만, 종가 기준으로는 한 달 동안 상승 방향이었습니다.

다만 "한 달 전"의 exact한 수치를 원하시면, 특정 날짜를 지정해 주셔야 정확하게 비교해 드릴 수 있습니다(예: 2026-03-25 기준 종가). 원하시는 날짜를 알려주시면 그날의 종가와 현재 종가를 비교해 차이를 계산해 드리겠습니다. 또한 원하시면 한 달 간의 최대 상승/하락 폭, 평균 거래량 같은 추가 지표도 함께 정리해 드릴게요.

메시지를 입력하세요



2. GPT와 미국 주식 이야기하기

- (실습) 종목 최근 주가 정보와 추천 정보 가져오기

7. streamlit run 파일명 .py 명령어로 코드를 실행

- 해당 주식의 매수를 추천하는지 묻기

```
ChatCompletionMessage(content=None, refusal=None, role='assistant',
annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_kPSoyO9YxYk
lpxX7SAV4G8ad', function=Function(arguments='{"ticker":"TSLA"}',
name='get_yf_stock_recommendations'), type='function')])
```

	period	strongBuy	buy	hold	sell	strongSell
0	0m	5	18	18	4	2
1	-1m	5	18	17	6	2
2	-2m	4	16	17	6	2
3	-3m	4	17	18	6	2



테슬라 매수 추천하니?



요청하신 매수 여부에 대해 단정적으로 말씀드리진 않겠지만, 현재 애널리스트들의 요약된 의견을 바탕으로 분위기를 정리해 드릴게요.

요약

- 0개월 데이터 기준(가장 최근): 매수 강도는 strong Buy 5, Buy 18으로 합계 23건, Hold 18건, Sell 4, Strong Sell 2로 총 47건. 매수 류의 비율은 약 49%, Hold가 38%, 매도류가 약 13% 정도로 혼재된 상황입니다.
- 1개월 전/2개월 전/3개월 전도 비슷한 흐름인데, 매수류가 4050%대, Hold가 약 3540%, 매도류가 8~13% 정도로 나타납니다.
- 즉, 애널리스트 의견은 “매수 가능성은 있지만 확고한 매수 신호만은 아니다”라는 중립에 가까운 편이고, 변동성도 여전히 존재하는 상황입니다.

해석 및 고려사항

- 장기 투자의 관점에서 보면 긍정적 모멘텀이나 성장 여력이 있다 여겨지는 날도 있지만, 단기간의 변동성 요인은 여전히 존재합니다.
- 테슬라는 산업 경쟁, 공급망 이슈, 브랜드 가치와 수요 변동 등 외부 요인에 의해 주가가 크게 움직일 수 있습니다.
- 따라서 단순히 애널리스트 추천만으로 결정하기보다는 본인의 투자 목표, 위험 허용도, 투자 기간을 함께 고려하는 것이 좋습니다.

추천 여부에 대한 간단한 판단 프레임

- 투자 기간이 길고 위험 허용도가 높은 편이라면, 분할매수(DCA)나 일정 비율의 포트폴리오 내 비중

메시지를 입력하세요



3. 스트림 출력하기

- 현재 응답은 GPT API에서 한 번에 완성해서 반환
- 답변이 나올 때까지 시간이 오래 걸림
- 스트림 출력
 - 챗GPT나 Bing(Bing)처럼 생성된 답변이 타이핑하듯이 출력

3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

1. GPT에서 스트림 출력을 확인할 수 있도록 stock_info_streamlit.py을 수정

```
... ( 생략 ) ...

def get_ai_response(messages, tools=None, stream=True):
    response = client.chat.completions.create(
        model="gpt-5-nano", # 응답 생성에 사용할 모델 지정
        stream=stream,      # 스트리밍 출력을 위해 설정
        messages=messages,  # 대화 기록을 입력으로 전달
        tools=tools,        # 사용 가능한 도구 목록 전달
    )
    if stream:
        for chunk in response:
            yield chunk # 생성된 응답의 내용을 yield로 순차적으로 반환합니다.
    else:
        return response # 생성된 응답의 내용을 반환합니다.

... ( 생략 ) ...
```

3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

2. ai_response 코드를 주석 처리하고 for 문을 이용해 ai_response를 출력

```
... ( 생략 ) ...

if user_input := st.chat_input():    # 사용자 입력 받기
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자 메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input) # 사용자 메시지를 브라우저에서도 출력

    ai_response = get_ai_response(st.session_state.messages, tools=tools)

    for chunk in ai_response:
        print(chunk)

    print('=====')

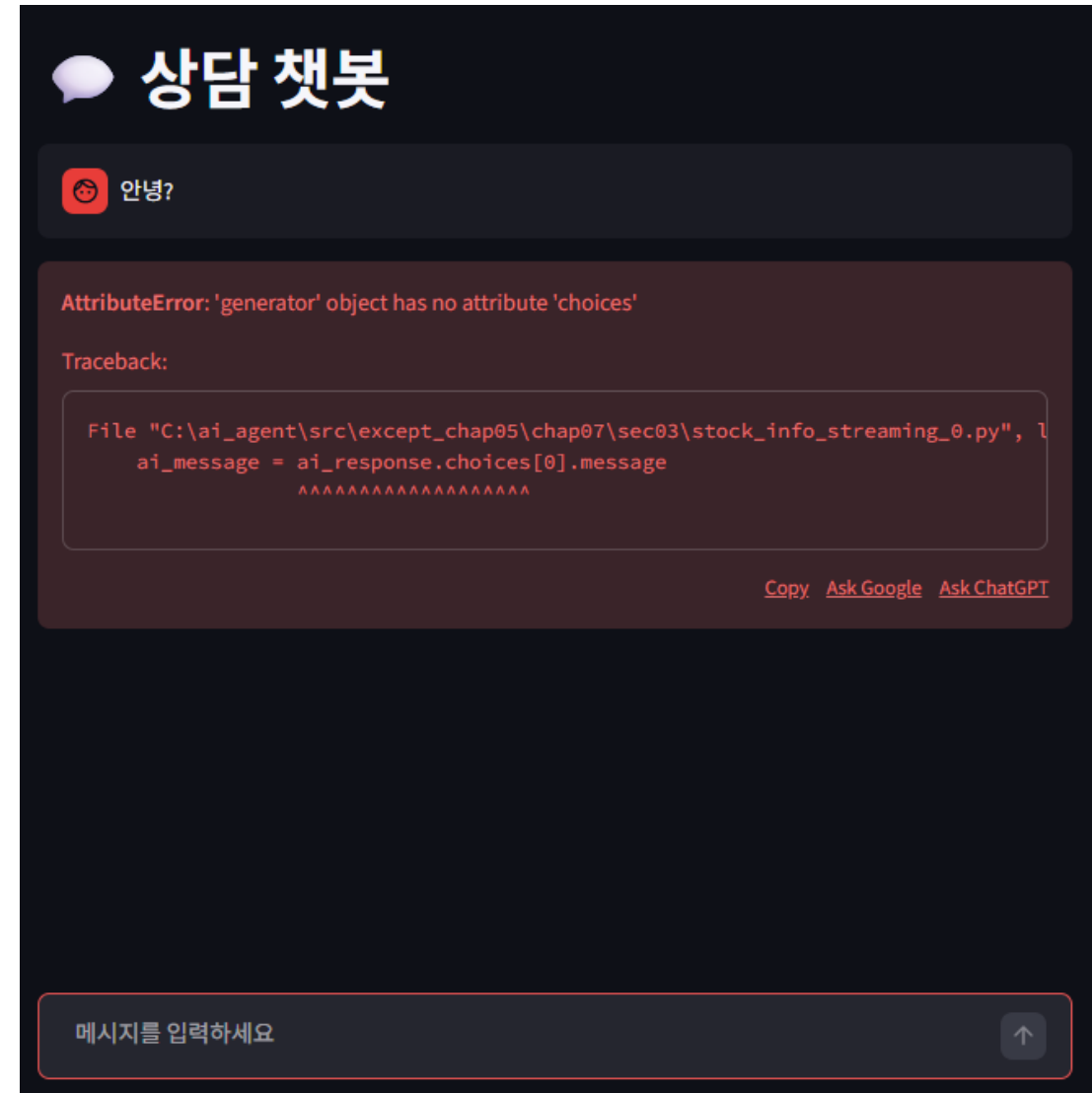
    ai_message = ai_response.choices[0].message
    # print(ai_message)

... ( 생략 ) ...
```

3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

3. 현재 상태에서 터미널 창에서 streamlit run 파일명 .py으로 코드를 실행하고 웹 브라우저에서 간단한 문장을 입력하면 오류 화면 발생



3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

4. 터미널 창에는 ChatCompletionChunk라는 객체로 결과 출력

```
ChatCompletionChunk(id='chatcmpl-DQDxOOoFySkiCsr2jHmzWRYuoOKOa', choices=[Choice(delta=ChoiceDelta(content='', function_call=None, refusal=None, role='assistant', tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1775142550, model='gpt-5-nano-2025-08-07', object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None, obfuscation='3a')
ChatCompletionChunk(id='chatcmpl-DQDxOOoFySkiCsr2jHmzWRYuoOKOa', choices=[Choice(delta=ChoiceDelta(content='안', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1775142550, model='gpt-5-nano-2025-08-07', object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None, obfuscation='jkZ')
ChatCompletionChunk(id='chatcmpl-DQDxOOoFySkiCsr2jHmzWRYuoOKOa', choices=[Choice(delta=ChoiceDelta(content='녕하세요', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1775142550, model='gpt-5-nano-2025-08-07', object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None, obfuscation='')
ChatCompletionChunk(id='chatcmpl-DQDxOOoFySkiCsr2jHmzWRYuoOKOa', choices=[Choice(delta=ChoiceDelta(content='!', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1775142550, model='gpt-5-nano-2025-08-07', object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None, obfuscation='pYG')
```

... (생략) ...

3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

5. 터미널 창에서 결과가 스트림 방식으로 출력되도록 코드 수정

... (생략) ...

```
if user_input := st.chat_input():    # 사용자 입력 받기
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자 메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input) # 사용자 메시지를 브라우저에서도 출력

    ai_response = get_ai_response(st.session_state.messages, tools=tools)
```

```
content = ''
for chunk in ai_response:
    content_chunk = chunk.choices[0].delta.content # 청크 속 content 추출
    if content_chunk:                             # 만약 content_chunk가 있다면,
        print(content_chunk, end="")             # 터미널에 줄바꿈 없이 이어서 출력
        content += content_chunk                 # content에 덧붙이기
```

```
print('\n=====')
print(content)
```

```
ai_message = ai_response.choices[0].message
# print(ai_message)
```

... (생략) ...

3. 스트림 출력하기

- (실습) 터미널 창에서 스트림 방식으로 출력하기

6. 현재 상태에서 streamlit run 파일명.py으로 실행('KPOP의 문제점을 이야기해 봐'라고 질문)

- 화면상에 여전히 오류는 발생하지만, 터미널 창에 스트림 방식으로 출력되는 결과 확인

K-pop 열풍은 여러 요인이 서로 맞물려 생긴 복합 현상이에요. 주요 원인을 정리해보면 다음과 같아요.

- 글로벌 스트리밍과 소셜 미디어의 힘
 - YouTube, Spotify 같은 플랫폼으로 한국 음악이 전 세계적으로 빠르게 확산되고, TikTok 같은 짧은 콘텐츠가 바이럴을 촉진합니다.
 - 팬덤이 온라인에서 활발히 활동하며 스트리밍과 뮤직비디오 조회수를 늘립니다.
- 고품질 음악과 퍼포먼스
 - 멜로디와 훅이 강하고 다양한 장르(팝, 힙합, EDM, R&B 등)를 융합한 트랙 구성.
 - 정교한 안무, 화려한 뮤직비디오, 시각적으로 매력적인 연출.
- 아이돌 시스템과 트레이닝 문화
 - 기획사 주도의 체계적인 연습생 양성, 팀 빌드와 브랜딩, 꾸준한 프로모션 활동으로 완성도 높은 퍼포먼스를 지속합니다.
 - 팀워크와 퍼포먼스의 일관된 퀄리티가 팬들의 만족도를 높입니다.
- 한류의 글로벌 네트워크와 협업
 - 드라마, 영화, 패션, 광고 등 다양한 콘텐츠와의 융합으로 시너지를 만듭니다.
 - 글로벌 아티스트와의 협업, 현지화 전략(다국어 자막, 지역 투어)으로 지역별 매력을 극대화합니다.

... (생략) ...

3. 스트림 출력하기

- (실습) 스트림릿에서 스트림 방식으로 출력하기

1. with를 사용해 비어 있는 챗 메시지를 만들고 그 메시지를 assistant로 설정

```
... ( 생략 ) ...

if user_input := st.chat_input():    # 사용자 입력 받기
    ... ( 생략 ) ...

    ai_response = get_ai_response(st.session_state.messages, tools=tools)

    content = ''
    tool_calls = None # tool_calls 초기화

    with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
        for chunk in ai_response:
            content_chunk = chunk.choices[0].delta.content # 청크 속 content 추출
            if content_chunk:                               # 만약 content_chunk가 있다면,
                print(content_chunk, end="")               # 터미널에 줄바꿈 없이 이어서 출력
                content += content_chunk                   # content에 덧붙이기
                st.markdown(content)                       # 스트림릿 챗 메시지에 마크다운으로 출력

    print('\n=====')
    print(content)

    ai_message = ai_response.choices[0].message
    # print(ai_message)

... ( 생략 ) ...
```

3. 스트림 출력하기

● (실습) 스트림릿에서 스트림 방식으로 출력하기

2. 코드를 실행하면 스트림릿에서 답변이 타이핑하듯 스트림 방식으로 출력되고 오류 발생

 **상담 챗봇**

 KPOP 열풍의 원인은?

 KPOP 열풍은 하나의 요인만으로 설명되기보다는 여러 요소가 서로 맞물린 결과입니다. 주요 원인들을 정리해 드리면 다음과 같아요.

- 산업적 전략과 구조
 - 아이돌 트레이닝 시스템: 어린 시절부터 다재다능한 멤버를 체계적으로 양성하고, 노래·댄스뿐 아니라 언어, 방송 태도까지 종합적으로 훈련합니다.
 - 소속사 중심의 글로벌 확장: 큰 기획사들이 해외 시장을 겨냥해 글로벌 투어, 현지 협업, 다국어 프로모션 등을 적극 추진합니다.
 - 콘텐츠 다각화: 뮤직비디오, 예능 형식의 리얼리티, 드라마틱한 컴백 스토리, 패션/뷰티 협업 등 음악 외 콘텐츠로 팬층을 넓힙니다.
- 기술과 디지털 플랫폼의 힘
 - 소셜 미디어와 영상 플랫폼 활용: 유튜브, 멜론/스포티파이 같은 음악 플랫폼, VLive 등의 라이브 스트리밍으로 글로벌 팬과 실시간 소통이 가능해졌습니다.
 - 팬덤 문화의 자발적 확산: 팬들이 자발적으로 콘텐츠를 번역·편집하고, 챌린지나 팬아트 등으로 활발히 참여해 확산력이 커집니다.
 - 알고리즘의 영향: 짧은 시간 안에 높은 조회수와 참여를 만들면 노출이 확대되어 더 많은 사람들에게 도달합니다.
- 음악적 매력과 창의성

메시지를 입력하세요 

원인들은 서로 영향을 주고받으며 시너지 효과를 냅니다. 특정 측면(예: 비즈니스 전략, 기술적 요인, 팬덤 심리)에서 더 자세히 알고 싶으시면 말씀해 주세요. 또한 KPOP을 건강하게 즐기고 싶은지, 아니면 산업 현황을 분석하고 싶은지에 따라 더 구체적인 정보를 맞춤으로 드릴 수 있습니다. 어떤 관점에서 더 알고 싶으신가요?

AttributeError: 'generator' object has no attribute 'choices'

Traceback:

```
File "C:\ai_agent\src\except_chap05\chap07\sec03\stock_info_streaming_1.py", line 1
    ai_message = ai_response.choices[0].message
                        ^^^^^^^^^^^^^^^^^^^^^^^
                        AttributeError
```

[Copy](#) [Ask Google](#) [Ask ChatGPT](#)

메시지를 입력하세요 

3. 스트림 출력하기

- (실습) 스트림릿에서 스트림 방식으로 출력하기

3. 코드가 맞지 않게 된 부분을 삭제하거나 주석 처리하고 영향을 받는 몇 가지 부분을 수정

```
... ( 생략 ) ...

content = ''
tool_calls = None

with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    ... ( 생략 ) ...

print('\n=====')
print(content)

# ai_message = ai_response.choices[0].message
# tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls를 가져옵니다.

if tool_calls:
    ... ( 생략 ) ...

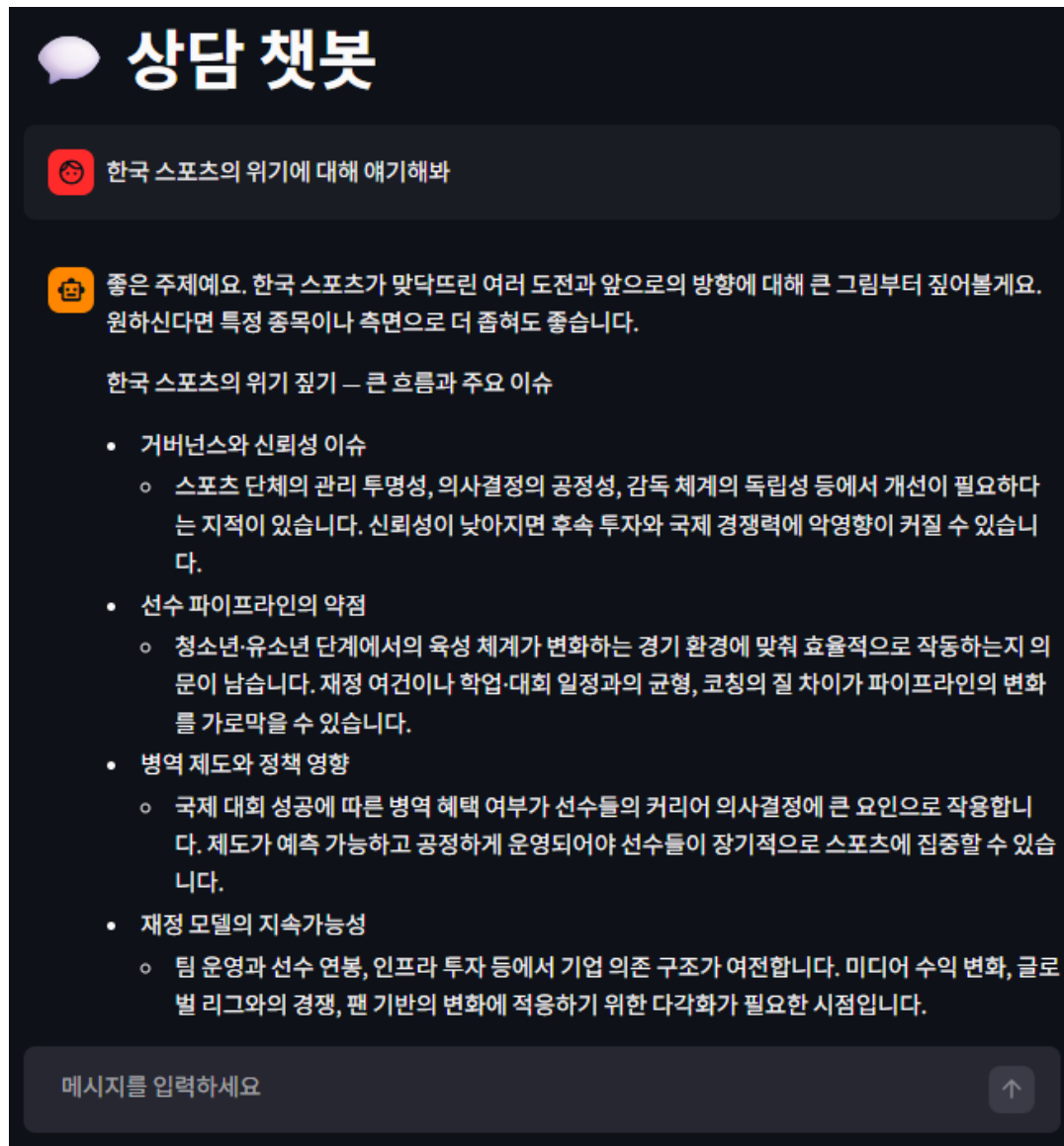
st.session_state.messages.append({
    "role": "assistant",
    "content": content # ai_message.content
}) # AI 응답을 대화 기록에 추가합니다.

# output = ai_message.content or ""
# print("AI\t:", output)
print("AI\t:", content)
# st.chat_message("assistant").write(output) # 브라우저에 메시지 출력
```

3. 스트림 출력하기

- (실습) 스트림릿에서 스트림 방식으로 출력하기

4. 코드가 맞지 않게 된 부분을 삭제하거나 주석 처리하고 영향을 받는 몇 가지 부분을 수정



3. 스트림 출력하기

● (실습) 스트림 방식에서 평선 콜링 사용하기

- stream이 True인 상태에서 평선 콜링을 사용하면 그 결과도 chunk 단위로 반환됨

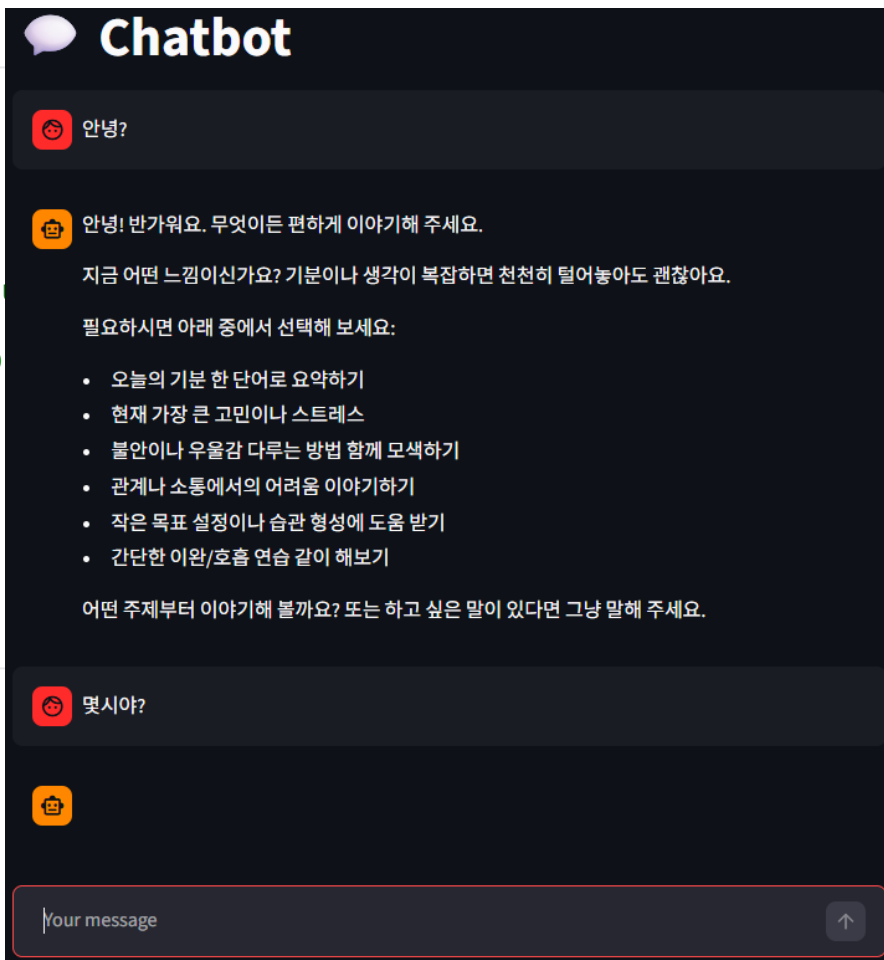
1. chunk를 출력하는 코드를 임시로 추가

... (생략) ...

```
with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content # 청크 속 content
        if content_chunk:                             # 만약 content_chunk가
            print(content_chunk, end="")              # 터미널에 줄바꿈
            content += content_chunk                  # content에 덧붙임
            st.markdown(content)                     # 스트림릿 챗 메시지
```

```
print(print(chunk))
print('\n=====')
print(content)
```

... (생략) ...

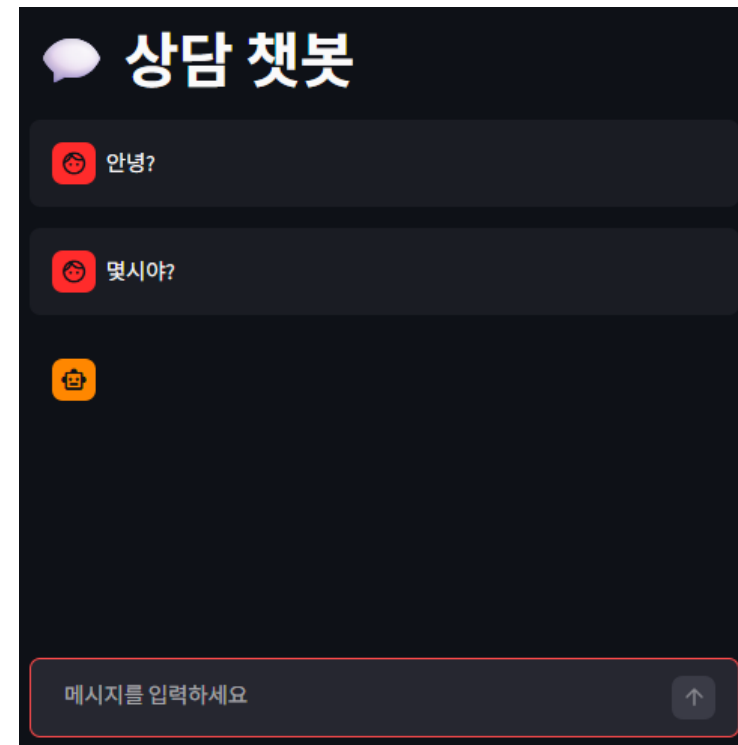


3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

2. 터미널에 출력된 결과

```
ChatCompletionChunk(id='chatcmpl-DQMipdL7eqibotu8SrBERKze0kosV',
choices=[Choice(delta=ChoiceDelta(content=None, function_call=None, refusal=None, role='assistant',
tool_calls=[ChoiceDeltaToolCall(index=0, id='call_nABRzPsjo71eu9mH5IEUsIaH',
function=ChoiceDeltaToolCallFunction(arguments='', name='get_current_time'), type='function'))],
finish_reason=None, index=0, logprobs=None)], created=1775176243, model='gpt-5-nano-2025-08-07',
object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None,
obfuscation='3uAFCKh1')
ChatCompletionChunk(id='chatcmpl-DQMipdL7eqibotu8SrBERKze0kosV',
choices=[Choice(delta=ChoiceDelta(content=None, function_call=None, refusal=None, role=None,
tool_calls=[ChoiceDeltaToolCall(index=0, id=None,
function=ChoiceDeltaToolCallFunction(arguments='{', name=None), type=None)], finish_reason=None,
index=0, logprobs=None)], created=1775176243, model='gpt-5-nano-2025-08-07',
object='chat.completion.chunk', service_tier='default', system_fingerprint=None, usage=None,
obfuscation='5N77YtO') ... ( 생략 ) ...
```



3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

3. 출력되는 정보를 한데 모으기 위해 코드 수정

... (생략) ...

```
tool_calls_chunk = [] # tool_calls_chunk 초기화
```

```
with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content # 청크 속 content 추출
        if content_chunk: # 만약 content_chunk가 있다면,
            print(content_chunk, end="") # 터미널에 줄바꿈 없이 이어서 출력
            content += content_chunk # content에 덧붙이기
            st.markdown(content) # 스트림릿 챗 메시지에 마크다운으로 출력
```

```
# print(chunk) # 임시로 청크 출력
if chunk.choices[0].delta.tool_calls: # tool_calls가 있는 경우
    tool_calls_chunk += chunk.choices[0].delta.tool_calls # tool_calls_chunk에 추가
```

```
print('\n=====')
print(content)
```

```
print('\n===== tool_calls_chunk') # tool_calls_chunk 확인하기 위한 코드
for tool_call_chunk in tool_calls_chunk:
    print(tool_call_chunk)
```

... (생략) ...

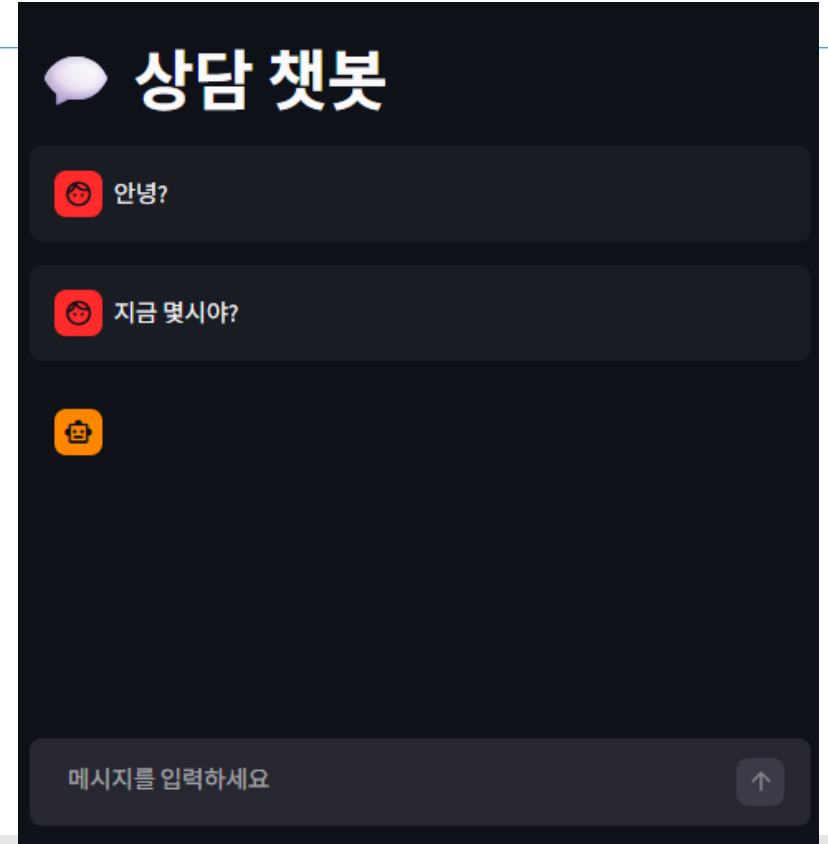
3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

4. 결과 확인

- 파일을 실행하고 다음처럼 get_current_time 함수를 실행하는 질문을 입력하면 해당 함수가 필요한 질문에는 여전히 출력되지 않음
- 하지만 터미널 창에 출력된 결과는 아래와 같음

- tool_calls_chunk 리스트에 어떤 함수를 실행해야 하는지와 그 인자는 어떻게 입력해야 하는지에 관한 정보가 쪼개진 청크 상태로 담겨있음
- 이 정보를 잘 합쳐서 get_current_time과 {"timezone":"Asia/Seoul"}을 만들어 내면 됨



===== tool_calls_chunk

```
ChoiceDeltaToolCall(index=0, id='call_FNEaZutA1NijTrK3VX3h4Rx7', function=ChoiceDeltaToolCallFunction(arguments='', name='get_current_time') type='function')
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='{', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='timezone', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='":', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='Asia', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='/', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='Se', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='oul', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='"}', name=None), type=None)
```

AI :

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

5. 조각난 정보를 모아 딕셔너리 형태로 정리하기 위해 tool_list_to_tool_obj 함수 만들기

- <https://community.openai.com/t/help-for-function-calls-with-streaming/627170/2> 오픈AI 커뮤니티 참조 작성
- 이 함수는 tools로 전달된 조각 단위의 정보를 리스트 형태로 받아 딕셔너리 형태의 리스트로 반환

```
from gpt_functions import get_current_time, tools, get_yf_stock_info, get_yf_stock_history, get_yf_stock_recommendations
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st
from collections import defaultdict

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def tool_list_to_tool_obj(tools):
    # 기본 값을 가진 딕셔너리 초기화
    tool_calls_dict = defaultdict(lambda: {"id": None, "function": {"arguments": "", "name": None}, "type": None})

    ... continue ...
```

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

5. 조각난 정보를 모아 딕셔너리 형태로 정리하기 위해 tool_list_to_tool_obj 함수 만들기

```
# 도구(함수) 호출을 반복하여 처리
for tool_call in tools:
    # id가 None이 아닌 경우 설정
    if tool_call.id is not None:
        tool_calls_dict[tool_call.index]["id"] = tool_call.id

    # 함수 이름이 None이 아닌 경우 설정
    if tool_call.function.name is not None:
        tool_calls_dict[tool_call.index]["function"]["name"] = tool_call.function.name

    # 인수 추가
    tool_calls_dict[tool_call.index]["function"]["arguments"] += tool_call.function.arguments

    # 타입이 None이 아닌 경우 설정
    if tool_call.type is not None:
        tool_calls_dict[tool_call.index]["type"] = tool_call.type

# 딕셔너리를 리스트로 변환
tool_calls_list = list(tool_calls_dict.values())

return {"tool_calls": tool_calls_list}
```

... (생략) ...

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

6. 함수 사용하기

```
... ( 생략 ) ...

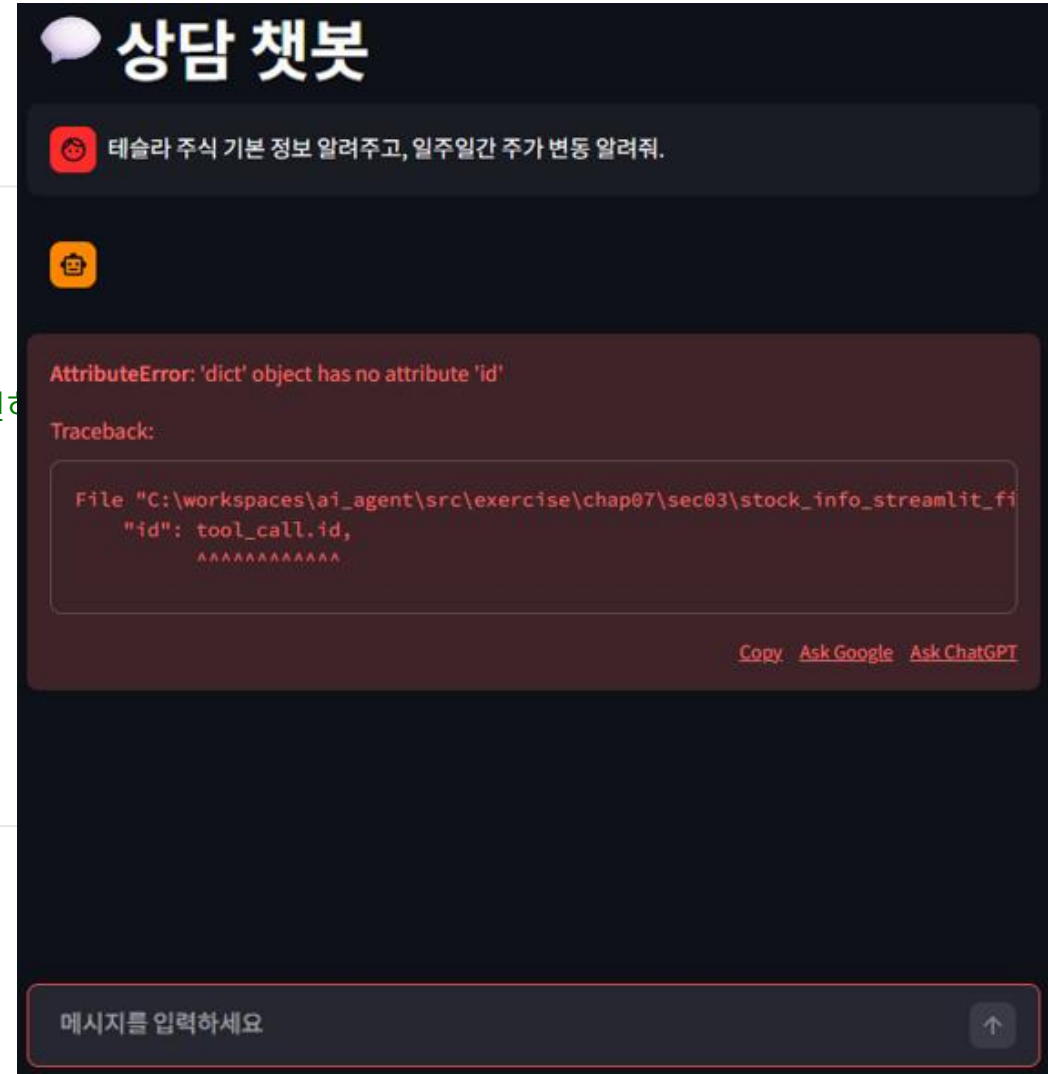
print('\n=====')
print(content)

# print('\n===== tool_calls_chunk') # tool_calls_chunk 확인
# for tool_call_chunk in tool_calls_chunk:
#     print(tool_call_chunk)

tool_obj = tool_list_to_tool_obj(tool_calls_chunk) # 위로 이동
tool_calls = tool_obj["tool_calls"] # 위로 이동
print(tool_calls)

if tool_calls:

... ( 생략 ) ...
```



3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

7. 터미널 창에 출력된 메시지 - 조각나 있던 평선 콜링 관련 정보들이 결합되어 있음

```
=====
```

```
[{'id': 'call_odCmah8jvA6sGs48l8W303H9', 'function': {'arguments': '{"ticker": "TSLA"}', 'name': 'get_yf_stock_info', 'type': 'function'}, {'id':  
'call_jaPK3ZF4w7hiqhWl9WmXCp5E', 'function': {'arguments': '{"ticker": "TSLA", "period": "5d"}', 'name': 'get_yf_stock_history', 'type': 'function'}}]
```

```
----- Traceback (most recent call last) -----
```

```
... ( 생략 ) ...
```

```
AttributeError: 'dict' object has no attribute 'id'
```

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

8. tool_calls가 있을 때 함수를 실행할 수 있도록 코드 수정

... (생략) ...

```
if tool_calls:
    st.session_state.messages.append({
        "role": "assistant",
        "content": None,
        "tool_calls": [
            {
                "id": tool_call["id"],
                "type": "function",
                "function": {
                    "name": tool_call["function"]["name"],
                    "arguments": tool_call["function"]["arguments"],
                }
            }
        ]
    })
    for tool_call in tool_calls
```

... continue ...

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

8. tool_calls가 있을 때 함수를 실행할 수 있도록 코드 수정

```
for tool_call in tool_calls:
    #tool_name = tool_call.function.name
    #tool_call_id = tool_call.id

    # 딕셔너리 형태에서 받기
    tool_name = tool_call["function"]["name"] # 실행해야한다고 판단한 함수명 받기
    tool_call_id = tool_call["id"] # 함수 아이디 받기

    try:
        #arguments = json.loads(tool_call.function.arguments)
        arguments = json.loads(tool_call["function"]["arguments"]) # 문자열을 딕셔너리로 변환
```

... (생략) ...

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

9. 함수 실행 후, 그 결과를 다시 get_ai_response 함수를 이용해 받아 오기

... (생략) ...

```
ai_response = get_ai_response(st.session_state.messages, tools=tools)
#ai_message = ai_response.choices[0].message # 수석 처리
content = ""
with st.chat_message("assistant").empty():
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content
        if content_chunk:
            print(content_chunk, end='')
            content += content_chunk
            st.markdown(content) # 스트림릿 챗메시지에 markdown으로 출력

st.session_state.messages.append({
    "role": "assistant",
    "content": content # ai_message.content
}) # AI 응답을 대화 기록에 추가합니다.
```

... (생략) ...

3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

10. 실행 결과

처음 `get_ai_response()` 함수를 실행할 때 평선 콜링이 필요한 경우에도 빈 메시지 칸을 생성하는데 그 칸에 `content`가 없어서 공백으로 남기 때문. 이 부분은 어떤 함수를 임시로 사용했는지 보여주는 용도로 활용해 봄



3. 스트림 출력하기

- (실습) 스트림 방식에서 평선 콜링 사용하기

- 11. 스트림릿에서 비어 있는 메시지에 함수 정보 넣기

```
... ( 생략 ) ...  
    # print(chunk) # 임시로 청크 출력  
    if chunk.choices[0].delta.tool_calls: # tool_calls가 있는 경우  
        tool_calls_chunk += chunk.choices[0].delta.tool_calls # tool_calls_chunk에 추가  
  
    tool_obj = tool_list_to_tool_obj(tool_calls_chunk)  
    tool_calls = tool_obj["tool_calls"]  
  
    if len(tool_calls) > 0: # 만약 tool_calls가 존재하면, st.write로 tool_call 내용 출력  
        print(tool_calls)  
        # tool_calls에서 function 정보만 모아서 출력  
        tool_call_msg = [tool_call["function"] for tool_call in tool_calls]  
        st.write(tool_call_msg)  
  
    print('\n=====')  
    print(content)  
  
    #tool_obj = tool_list_to_tool_obj(tool_calls_chunk) # 위로 이동  
    #tool_calls = tool_obj["tool_calls"] # 위로 이동  
    #print(tool_calls)  
  
    if tool_calls:  
  
... ( 생략 ) ...
```

3. 스트림 출력하기

- (실습) 스트림 방식에서 평션 콜링 사용하기

12. 실행 결과

